



자연어처리와 컴퓨터비전 심층학습 6장(Word2Vec~)

[Deep Learning] 자연어처리와 컴퓨터비전 심층학습: Word2Vec부터 CNN까지

자연어처리(NLP)와 컴퓨터비전(CV)은 딥러닝이 가장 눈부신 성과를 이뤄낸 두 축입니다. 단어를 어떻게 숫자로 표현할 것인가에 대한 고민에서 출발한 Word2Vec부터, 순차적 데이터를 처리하는 RNN, 그리고 공간적 특징을 추출하는 CNN까지 심층학습의 핵심 모델들을 정리해 보았습니다.

파트(A): 단어의 의미를 벡터 공간에 매핑하다, Word2Vec (1)

1. 왜 Word2Vec이 등장하게 되었을까? (vs. TF-IDF)

딥러닝 이전, 자연어처리에서 단어를 벡터화하는 대표적인 방법은 원-핫 인코딩(One-hot Encoding)과 이를 기반으로 문서 내 단어의 중요도를 평가하는 **TF-IDF(Term Frequency-Inverse Document Frequency)**였습니다.

TF-IDF는 정보 검색에서 훌륭한 성능을 냈지만, 치명적인 한계가 있었습니다.

- **희소 문제(Sparsity):** 단어 사전의 크기가 커질수록 벡터의 차원이 기하급수적으로 커지며, 대부분의 값이 0인 희소 벡터가 됩니다.
- **의미적 유사도 반영 불가:** 원-핫 인코딩 기반이므로 모든 단어 벡터가 서로 직교(Orthogonal)합니다. 즉, '강아지'와 '개'의 내적이 0이 되어, 단어 간의 의미적 유사성을 전혀 파악할 수 없습니다.

이를 해결하기 위해 등장한 것이 단어를 고정된 크기의 조밀한 벡터(Dense Vector)로 표현하는 **워드 임베딩(Word Embedding)**이며, 그 패러다임을 이끈 모델이 바로 **Word2Vec**입니다. Word2Vec은 "비슷한 위치에서 등장하는 단어들은 비슷한 의미를 가진다"는 분포적 가설(Distributional Hypothesis)을 기반으로 신경망을 통해 단어 벡터를 학습합니다.

2. CBOW와 Skip-gram

Word2Vec의 학습 방법론은 크게 두 가지로 나뉩니다.

- **CBOW (Continuous Bag of Words)**: 주변에 있는 단어(Context)들을 입력으로 받아 중심 단어(Target)를 예측하는 방식입니다. 주변 단어들의 벡터를 평균 내어 예측하므로 학습 속도가 빠르지만, 문맥이 뭉개지는 경향이 있어 등장 빈도가 낮은 단어에는 취약합니다.
- **Skip-gram**: 중심 단어(Target)를 입력으로 받아 주변 단어(Context)들을 예측하는 방식입니다. 하나의 중심 단어에서 여러 번의 업데이트가 발생하므로 연산량은 많지만, 단어의 미묘한 의미 차이를 잘 포착하며 특히 **희귀 단어(Rare words) 처리에 강력**하여 실제 연구에서는 Skip-gram이 주로 사용됩니다.

3. 학습 최적화 기법: 계층적 Softmax와 Negative Sampling

수십만 개의 단어 사전을 상대로 일반적인 Softmax 연산을 수행하면 계산 비용이 너무 큽니다. 이를 해결하기 위한 두 가지 기법이 도입되었습니다.

- **계층적 Softmax (Hierarchical Softmax)**: 단어 사전을 이진 트리(Binary Tree) 형태로 구성하여, 탐색 시간 복잡도를 $O(V)$ 에서 $O(\log_2 V)$ 로 줄입니다.
- **네거티브 샘플링 (Negative Sampling)**: 전체 단어 집합이 아닌, 정답인 주변 단어(Positive) 1개와 오답인 임의의 단어(Negative) K 개만을 샘플링하여 이진 분류(Binary Classification) 문제로 바꾸어 계산 효율을 극대화합니다.

💡 [코드 설명] Skip-gram 기반 Word2Vec 구조 (PyTorch)

```
import torch
import torch.nn as nn

class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_size):
        super(SkipGram, self).__init__()
        # 중심 단어 임베딩
        self.in_embed = nn.Embedding(vocab_size, embed_size)
        # 주변 단어 임베딩
        self.out_embed = nn.Embedding(vocab_size, embed_size)
```

```

def forward(self, target, context, negative_words):
    # 정답(Positive)에 대한 내적 (최대화 목표)
    target_vec = self.in_embed(target) # [batch, embed_size]
    context_vec = self.out_embed(context) # [batch, embed_size]
    pos_score = torch.sum(target_vec * context_vec, dim=1)

    # 오답(Negative)에 대한 내적 (최소화 목표)
    neg_vec = self.out_embed(negative_words) # [batch, k, embed_size]
    neg_score = torch.bmm(neg_vec, target_vec.unsqueeze(2)).squeeze()

    return pos_score, neg_score

```

드 리뷰: PyTorch로 구현한 기본적인 Negative Sampling 방식의 Skip-gram입니다. 정답 단어와의 내적 점수는 높이고, 샘플링된 오답 단어들과의 내적 점수는 낮추는 방향으로 두 임베딩 층(`in_embed`, `out_embed`)의 가중치가 업데이트됩니다.

파트(B): Word2Vec의 확장과 순차 데이터의 도입

1. Word2Vec (2) - Gensim을 활용한 실습

실제 프로젝트에서는 Word2Vec을 직접 구현하기보다 최적화된 라이브러리인 `gensim` 을 주로 활용합니다.

```

from gensim.models import Word2Vec

# 문장 데이터 단위로 토큰화된 리스트 입력
model = Word2Vec(sentences=tokenized_data, vector_size=100, w
indow=5, min_count=5, sg=1) # sg=1: Skip-gram
print(model.wv.most_similar('인공지능'))

```

2. fastText: 서브워드(Subword)로 OOV를 정복하다

왜 fastText가 나오게 되었는지? (vs. Word2Vec)

Word2Vec은 훌륭하지만 '사전에 없는 단어(Out-Of-Vocabulary, OOV)'를 만나면 연산을 할 수 없고, 오타나 형태소 변화(동사 활용 등)에 매우 취약합니다.

Facebook(현 Meta)에서 발표한 **fastText**는 단어를 쪼갤 수 없는 최소 단위로 보지 않고, 단어 내부의 **n-gram (서브워드)** 들의 조합으로 바라봅니다.

예를 들어 `apple` 을 3-gram으로 나누면 `<ap, app, ppl, ple, le>` 와 중심 단어 `<apple>` 로 분리하여 임베딩한 뒤 이를 모두 더하여 단어 벡터를 생성합니다.

- **장점:** 훈련 데이터에 없던 단어(OOV)라도 서브워드가 학습되어 있다면 벡터를 추론할 수 있으며, 오타에 강건하고 한국어처럼 교착어 특성을 가진 언어에 매우 효과적입니다.

```
from gensim.models import FastText

# fastText 모델 학습 (Gensim 활용)
ft_model = FastText(sentences=tokenized_data, vector_size=100, window=5, min_count=5, sg=1)
# 훈련 데이터에 없는 신조어라도 유사도 도출 가능
print(ft_model.wv.most_similar('딥러닝최고'))
```

3. 순환 신경망(RNN)의 기초

단어를 임베딩했다면, 이제 이 단어들이 모인 '문장(시퀀스)'의 문맥을 처리할 신경망이 필요합니다.

- **순환 신경망(RNN, Recurrent Neural Network):** 은닉층의 노드에서 활성화 함수를 통해 나온 결과값을 출력층 방향으로도 보내면서, 동시에 다시 은닉층 노드의 다음 계산의 입력으로 보내는 특징을 갖습니다. 이전 시점의 정보(과거의 기억)를 현재 시점의 처리에 반영합니다.
- **다양한 구조:** * *One-to-Many*: 이미지 캡셔닝
 - *Many-to-One*: 감성 분류 (스팸 문자 판별 등)
 - *Many-to-Many*: 기계 번역, 챗봇

파트(C): 장기 기억을 위한 혁신, 순환 신경망(2)

1. 장단기 메모리 (LSTM, Long Short-Term Memory)

전통적인 Vanilla RNN은 시퀀스가 길어질수록 과거의 정보가 마지막까지 전달되지 못하는 **장기 의존성 문제(Long-Term Dependency)** 와 **기울기 소실(Vanishing Gradient)** 문제를 겪습니다.

이를 해결하기 위해 등장한 **LSTM**은 은닉 상태(Hidden State) 외에 **셀 상태(Cell State, C_t)** 라는 정보의 컨베이어 벨트를 추가했습니다. 그리고 이 벨트에 정보를 얼마나 올리고 뺄지를 3가지 '게이트(Gate)'를 통해 조절합니다.

- **망각 게이트(Forget Gate)**: 과거 정보를 얼마나 버릴지 결정
- **입력 게이트(Input Gate)**: 현재 정보를 얼마나 저장할지 결정
- **출력 게이트(Output Gate)**: 다음 은닉 상태로 어떤 값을 내보낼지 결정

💡 [코드 설명] LSTM 모델 실습 (PyTorch)

```
class TextClassifierLSTM(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, output_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim)
        # batch_first=True: 입력 텐서의 첫 차원을 배치 크기로 설정
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)

    def forward(self, text):
        embedded = self.embedding(text) # [batch, seq_len, embed_dim]
        lstm_out, (hidden, cell) = self.lstm(embedded)
        # 마지막 시점(Time Step)의 은닉 상태를 분류기에 전달 (Many-to-One)
        dense_outputs = self.fc(hidden[-1])
        return dense_outputs
```

코드 리뷰: `nn.LSTM` 은 내부에 복잡한 게이트 연산을 캡슐화하고 있습니다. 리턴값 중 `lstm_out` 은 모든 시점의 은닉 상태를, `hidden` 은 마지막 시점의 은닉 상태를 담고 있습니다. 감성 분석과

같은 문서 분류 태스크에서는 주로 마지막 상태인 `hidden[-1]` 을 Fully Connected Layer(`fc`)에 넘겨 예측을 수행합니다.

파트(D): 공간적 패턴을 캡처하다, 합성곱 신경망 (CNN)

1. 6.7 CNN의 주요 구성 요소

CNN은 본래 컴퓨터 비전을 위해 고안되었으나, 자연어 처리에서도 국소적인 단어 패턴(n-gram)을 추출하는 데 탁월한 성능을 보입니다.

- **합성곱 계층 (Convolution Layer):** 필터(커널)를 슬라이딩하며 데이터의 특징(Feature)을 추출합니다.
- **활성화 맵 (Activation Map):** 합성곱 연산과 활성화 함수(ReLU 등)를 거쳐 생성된 특징 지도입니다.
- **풀링 (Pooling):** 차원을 축소하여 파라미터를 줄이고, 노이즈를 상쇄하며 모델의 추상화 능력을 높입니다 (주로 Max Pooling 사용).
- **완전연결계층 (FCN, Fully Connected Network):** 추출된 특징들을 1차원으로 평탄화(Flatten)하여 최종적인 분류/예측을 수행합니다.

💡 [코드 설명] CNN 모델 실습 (PyTorch)

```
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Conv2d: in_channels, out_channels, kernel_size
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=
1)

        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 14 * 14, 10) # 28x28 이미지

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
```

기준

```
x = x.view(-1, 32 * 14 * 14) # Flatten 연산
x = self.fc1(x)
return x
```

코드 리뷰: 이미지 특징 추출의 기본이 되는 구조입니다. `nn.Conv2d`를 통해 지역적 패턴을 찾고, `nn.MaxPool2d`로 크기를 절반으로 줄인 뒤, `view`를 이용해 평탄화하여 `Linear` 층으로 넘겨 클래스 확률을 계산합니다.

2. CNN vs RNN: Inductive Bias의 관점에서 비교

왜 자연어는 주로 RNN으로, 이미지는 주로 CNN으로 처리할까요? 이는 두 네트워크가 가지고 있는 **귀납적 편향(Inductive Bias)**의 차이 때문입니다. (Inductive Bias란 학습 알고리즘이 만나보지 못한 입력에 대해 예측할 때 사용하는 추가적인 가정입니다.)

- **CNN의 Inductive Bias (지역성과 이동 불변성):**

- *Locality (지역성):* 이미지는 픽셀 단위로 볼 때, 인접한 픽셀끼리 강한 상관관계를 가진다고 가정합니다. 필터가 특정 윈도우 크기만큼만 스캔하는 이유입니다.
- *Translation Invariance (이동 불변성):* 고양이가 이미지 좌측 상단에 있든, 우측 하단에 있든 똑같은 '고양이' 특징으로 인식해야 합니다. 가중치를 공유하는 커널과 풀링 연산이 이를 가능하게 합니다.

- **RNN의 Inductive Bias (순차적 의존성):**

- *Sequentiality (순차성):* 데이터의 시간적 순서나 위치가 의미를 결정하는 데 매우 중요하다고 가정합니다. "I am" 뒤에 "hungry"가 오는 것은 앞의 단어 정보에 강하게 의존하기 때문입니다. 시간축을 따라 은닉 상태를 공유함으로써 시퀀스 데이터를 모델링합니다.

결론적으로, 데이터가 공간적 패턴과 독립적인 특징 모음이라면 CNN이, 시간적 흐름과 이전 상태가 맥락을 결정짓는다면 RNN이 강력한 성능을 발휘하게 됩니다.